



Final Project Report

Group 1: Max Bacharach and Erik Langemalm

DV1663 – Database Systems

Report Managers: Ilir Jusufi and Birgitta Hermanson

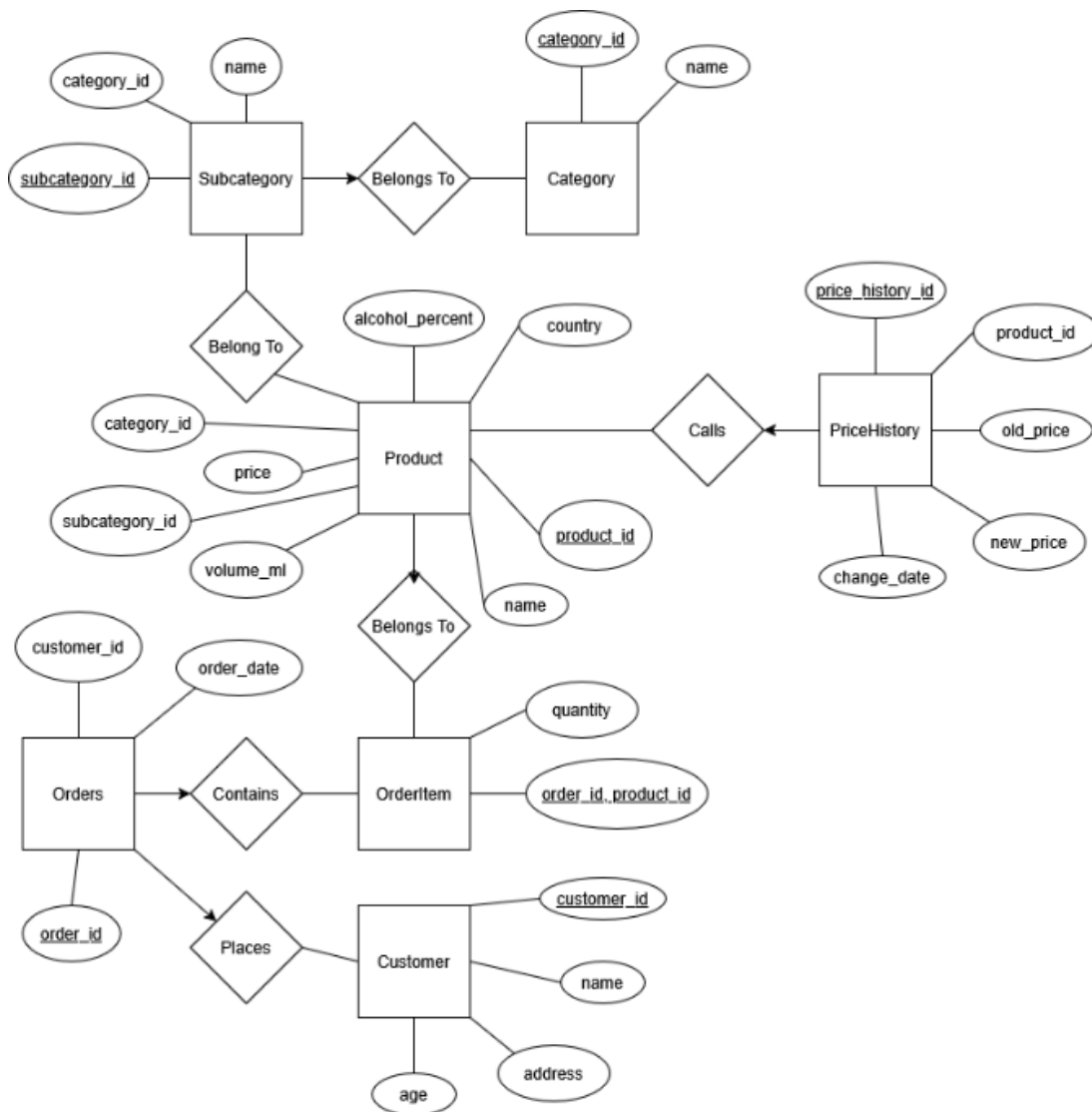
1. Project Idea

Our idea for this project was to create a database for the different types of alcoholic beverages a customer can put in a cart and place an order to get them shipped, like a Systembolaget website. In the database you can search/click your way forward to specific categories and subcategories to filter what type of beverages you might want to place an order on and then add them to a cart, which you are going to be able to access and see the quantity and what beverages you have putted in the cart.

The database should also, with the information from order, be able to count the average price for each category and even make a list of popular order items which users have bought. It should also keep a record of the history of the price for each item and how it has changed over time. The database system should also have a trigger to update the price history of a product.

2. Schema Design

- ER Diagram for the Database:



- Flowchart over the ER Diagram



3. SQL Queries

These are the following triggers, procedures and aggregations that we use in this project. Note that all these triggers are stored in the file Project.sql

1. Trigger update_price_history: This trigger affects the table PriceHistory where when a price is updated in the table product. Before the new price is set in the table products, the trigger runs and saves the old price in the PriceHistory table with the right product_id, sets the new price as well and saves which date and time the change was made. This trigger sequentially goes over each affected row and checks if the old price and new price are not equal. This ensures that only new prices that are different from the old ones are recorded.

```

DELIMITER $$

create trigger update_price_history
before update on Product
for each row
begin
    if new.price != old.price then
        insert into PriceHistory (product_id, old_price, new_price, change_date)
        values (old.product_id, old.price, new.price, now());
    end if;
end $$

DELIMITER ;
  
```

2. Procedure OldAndNewOrders: This procedure returns the old and the new orders placed separated in two datasets that allow us to separate old orders and new orders placed by a certain user. The procedure takes the customer_id as an input variable and then is used to filter which orders that user has made based on the date of the orders.

```
delimiter %%
create procedure OldAndNewOrders(in customer int)
begin
#Orders in process
select orderitem.quantity, product.product_id from orderitem join Orders on Orders.order_id = orderitem.order_id
join product on orderitem.product_id = product.product_id where Orders.order_date > curdate() and Orders.customer_id = customer;
#Old orders
select orderitem.quantity, product.product_id from orderitem join Orders on Orders.order_id = orderitem.order_id
join product on orderitem.product_id = product.product_id where Orders.order_date < curdate() and Orders.customer_id = customer;
end %%
```

3. View ProductSales: This query creates the view ProductSales that shows each product ID and name. It also calculates the total quantity sold per product and calculates the total revenue per product sold. It joins the Product and OrderItem tables and grouping the results by product.

```
create view ProductSales as
select p.product_id, p.name, sum(oi.quantity) as total_quantity_sold,
sum(p.price * oi.quantity) as total_revenue
from Product p
join OrderItem oi on p.product_id = oi.product_id
group by p.product_id, p.name;
```

4. Select sales per month: This query extracts the month from order_date and formats it as YYYY-MM and calculates the total sales for each month. It then joins the Orders, OrderItem and Product tables and groups the results by month and orders the results by month. Can be used to analyze monthly sales.

```
select date_format(o.order_date, '%Y-%m') as month, sum(p.price * oi.quantity) as total_sales
from Orders o
join OrderItem oi on o.order_id = oi.order_id
join Product p on oi.product_id = p.product_id
group by month
order by month;
```

5. Select Most sold product: This SQL query lists each product name and the total quantity sold and joins the Product and OrderItem tables. It then groups the results by product name and orders products by total quantity sold in descending order and returns only the top 10 best-selling products.

```
select p.name, sum(oi.quantity) as total_quantity_sold
from Product p
join OrderItem oi on p.product_id = oi.product_id
group by p.name
order by total_quantity_sold desc
limit 10;
```

4. Discussion and Resources

One of the early problems we faced during the project was the implementation of Flask in Python. We had a hard time connecting and running the website generated by Flask. The solution: Flask was installed in the wrong place which when we moved over the .venv environment the module we imported for flask to run in the cmd window couldn't recognize flask as a command. The flask command itself couldn't also be run in the preselected powershell window. Instead, we had to start a new cmd window from the powershell window, which allowed us to start the flask app. This was not something that was included in the documentation from the Flask website and had to be found externally.

Another issue we faced was the connection between Python and our SQL Database but we soon realized, through testing and searching for errors in the code, that we needed to change the SQL connection query in Python and that we needed to run the SQL queries once in MySQL Workbench in order to get any data to our website. We also used different templates to make sure that every query in Python Flask to work properly with the triggers and aggregations in MySQL and accessed our database.

In overall, the project has run smoothly for us with the help of Canvas practical assignments, Teacher to student conversations, AI, W3School and general google searches if it were any uncertainties in the project or the implementation of our database system.

The resources we used in this project can be downloaded following the instructions in the readme.md file on our Github. The readme.md includes an installation guide for all our dependencies used in this project, information about the project, setting up the environment for the project and instructions to connect to your database.

Github link: <https://github.com/Vixen-School/Project-Systembolaget>